A PROJECT REPORT ON

EMBERVAULT: THE CRYSTAL MAZE

A 2D Computer Graphics Cave-Exploration Game & Interactive Graphics Laboratory

---

Submitted in partial fulfillment of the requirements for the award of the degree of
BACHELOR OF TECHNOLOGY / BACHELOR OF ENGINEERING
IN
INFORMATION TECHNOLOGY / COMPUTER SCIENCE & ENGINEERING

Submitted by:
[STUDENT NAME]
Roll No. / Registration No.: [ROLL NUMBER]

Under the Guidance of:
[PROFESSOR / GUIDE NAME]
[Designation of Guide]
Department of [Department Name]

DEPARTMENT OF [DEPARTMENT NAME]
[INSTITUTION / UNIVERSITY NAME]
[CITY, STATE, PIN CODE]

ACADEMIC YEAR: [2025 – 2026]

CERTIFICATE OF APPROVAL

This is to certify that the project entitled **"EmberVault: The Crystal Maze"** is a bona fide record of work carried out by **[STUDENT NAME]** (Roll No: **[ROLL NUMBER]**) in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology / Engineering in [Department Name]** from **[Institution / University Name]** during the academic year **[2025 – 2026]**.

The project report has been approved as it satisfies the academic requirements prescribed for the Computer Graphics Laboratory and Theory curriculum.

_____

[GUIDE NAME]
Project Guide
Department of [Department]
[Institution Name]

_____

[HEAD OF DEPARTMENT]
Head of Department
Department of [Department]
[Institution Name]

_____

INTERNAL EXAMINER
Date:

_____

EXTERNAL EXAMINER
Date:

ACKNOWLEDGEMENTS

I express my deepest sense of gratitude and sincere thanks to my project guide, **[PROFESSOR / GUIDE NAME]**, [Designation], Department of [Department Name], [Institution Name], for invaluable guidance, encouragement, and constructive feedback throughout the design, development, and documentation of this Computer Graphics project.

I am also thankful to **[HEAD OF DEPARTMENT NAME]**, Head of the Department of [Department Name], for providing the academic infrastructure and computing laboratories required to complete this work.

Finally, I wish to thank my family, peers, and faculty members who directly or indirectly extended their support and assistance in completing **EmberVault: The Crystal Maze**.

[STUDENT NAME]
Roll No: [ROLL NUMBER]
Date: [DATE OF SUBMISSION]

ABSTRACT

The rapid evolution of modern video games and graphics software has distanced students from foundational rasterization and computational geometry algorithms. Modern game engines treat graphics as black-box abstractions, preventing undergraduate engineers from directly interacting with scanline converters, line clipping algorithms, and affine transformation matrices.

This project, **"EmberVault: The Crystal Maze,"** addresses this educational gap by developing a full-featured 2D computer graphics cave-exploration game and an interactive academic Computer Graphics Laboratory implemented entirely from mathematical scratch in Python using Pygame strictly for low-level pixel buffer display.

The software implements the **Bresenham Line Algorithm** and **Digital Differential Analyzer (DDA)** for line rasterization, the **Midpoint Circle Algorithm** for outer key halos and scanline-filled circular primitives, **Cohen-Sutherland Line Clipping** and **Liang-Barsky Line Clipping** to dynamically clip cave geometry against a moving rectangular torch-light window, and a complete **3x3 Homogeneous Matrix Transformation Pipeline** for translation, rotation, scaling, reflection, and shear.

Gameplay encompasses 5 procedurally generated, loop-injected caverns with circle-versus-cell pushout collision detection, dynamic fog of war, and intelligent enemy wraiths using Breadth-First Search (BFS) grid pathfinding. Adhering to a strict **Zero-Timer Design**, all cooldowns and ability charges operate on discrete gameplay actions rather than countdown clocks. An integrated **Computer Graphics Laboratory mode** allows interactive examination of discrete pixel grids, decision parameters, and 8-way circle symmetry. The system is verified through an automated 16-test headless verification suite passing with 100% success.

CHAPTER 1: INTRODUCTION

1.1 Project Title & Background

The project is titled **EmberVault: The Crystal Maze**. It is an interactive 2D cave-exploration maze game and academic graphics laboratory built entirely in Python using the Pygame library as a direct pixel buffer. The project bridges the gap between theoretical Computer Graphics coursework and practical software engineering by implementing classical algorithms—such as incremental line drawing, midpoint circle generation, parametric and outcode clipping, and homogeneous matrix transformations—within an active, playable game engine.

1.2 Problem Statement

In conventional undergraduate curricula, Computer Graphics is frequently taught as pure mathematical theory on paper or through commercial game engines where rendering details are hidden. Students rarely experience how an actual line is clipped against a camera view frustum or how a scanline filler converts an arbitrary polygon into display rows. Furthermore, many student games rely on artificial countdown timers that introduce frustrating time pressures rather than encouraging strategic, action-based problem solving.

1.3 Proposed Solution & Objectives

EmberVault solves this educational challenge by providing: (1) Mathematical Implementation from Scratch: Line drawing (DDA, Bresenham), circle generation (Midpoint), clipping (Cohen-Sutherland, Liang-Barsky), and transformations (3x3 matrices) written without high-level drawing libraries; (2) Dynamic Torch as an Active Clip Window: The game's torch is an active rectangular viewing window that clips all cavern geometry in real time; (3) Interactive Computer Graphics Laboratory: An academic mode accessible from the main menu where students can interactively manipulate endpoints, view live decision parameters, study 8-way circle symmetry in color, and examine 4-bit region outcodes; (4) Zero-Timer Gameplay Design: All abilities and mechanics recharge via player actions rather than countdown clocks.

CHAPTER 2: PROJECT OVERVIEW

2.1 Game Narrative & User Experience

The player assumes the role of an expedition explorer lost within the subterranean depths of EmberVault, a forgotten sanctum carved into obsidian rock. The environment is pitch black; the explorer holds a crystalline torch that illuminates a localized region of the cave. To escape, the explorer must locate hidden spinning amber keys, avoid or destroy hostile Shadow Wraiths, and unlock the celestial exit portal to progress through five increasingly intricate caverns.

2.2 Core Gameplay Loop

The gameplay loop consists of four interconnected phases: (1) Exploration & Fog of War: Moving through corridors while the torch light illuminates walls and entities, recording visited paths in a persistent discovery grid; (2) Combat & Evasion: Hostile wraiths patrol corridors while Stalker wraiths run BFS pathfinding to pursue the player, who can shoot arrows or activate an Energy Shield to absorb hits; (3) Resource Collection: Collecting amber keys, Cyan Surge crystals, and Violet Aegis crystals; (4) Portal Unsealing: Once all keys are collected, the portal activates, evaluating level completion and awarding badges.

CHAPTER 3: TECHNOLOGIES USED

3.1 Core Technologies Specification

The project relies on Python 3.10+ as the foundational programming language, chosen for its clean syntax, rapid prototyping, and high-precision integer arithmetic (preventing integer overflow in raster decision parameters). Pygame (v2.6.1) is used strictly as a lightweight wrapper for opening the OS window, capturing keyboard/mouse events, and drawing raw pixels or 1-pixel scanline spans (Surface.fill). No pre-built vector collision, physics, or geometry drawing functions from Pygame were used for game entities.

3.2 Storage & Standard Libraries

Local data persistence is achieved via flat text file storage (best_score.txt) using standard Python I/O with automatic fallback to in-memory tracking. Standard library modules include math (trigonometry, hypot), random (seeded procedural carving), collections.deque (FIFO queue for BFS), and os/sys (system paths and environment management).

CHAPTER 4: COMPUTER GRAPHICS THEORY

4.1 Pixels, Raster Grids, and Coordinate Systems

A pixel (picture element) is the smallest addressable unit of color on a display screen. In computer graphics displays, the screen origin (0, 0) is positioned at the top-left corner. The horizontal axis (X) increases from left to right (0 <= X <= W-1), while the vertical axis (Y) increases from top to bottom (0 <= Y <= H-1). This orientation contrasts with Cartesian mathematics where Y increases upward, requiring y-inversion adjustments during mathematical transformations.

4.2 Rasterization & Affine Transformations

Rasterization is the computational process of converting continuous geometric primitives defined by mathematical vector formulas into discrete pixels aligned with the display grid. In 2D Cartesian space, rotation and scaling are matrix multiplications, but translation is an addition. To unify all transformations into composable matrix products, Computer Graphics utilizes homogeneous coordinates, representing 2D points as 3D column vectors [x, y, 1]^T, allowing translation, rotation, scaling, reflection, and shear to be multiplied as 3x3 matrices.

CHAPTER 5: COMPUTER GRAPHICS ALGORITHMS

5.1 Bresenham's Integer Line Drawing Algorithm

Implemented in graphics.py:bresenham_line(). Bresenham's algorithm avoids floating-point divisions by using integer arithmetic and an incremental decision parameter p_k. For shallow slopes (dx > dy): initial decision parameter p_0 = 2*dy - dx. If p_k < 0, next pixel is (x+1, y) and p_{k+1} = p_k + 2*dy; if p_k >= 0, next pixel is (x+1, y+1) and p_{k+1} = p_k + 2*(dy - dx). Time complexity is O(max(dx, dy)).

5.2 Digital Differential Analyzer (DDA) Algorithm

Implemented in graphics.py:dda_line(). DDA calculates dx and dy, determines steps as max(|dx|, |dy|), computes floating increments x_inc = dx/steps and y_inc = dy/steps, and rounds (x, y) to integer pixels on each step. It is slower than Bresenham due to floating-point additions and round() calls.

5.3 Midpoint Circle Algorithm & 8-Way Symmetry

Implemented in graphics.py:circle_points(). Evaluates circle function f(x, y) = x^2 + y^2 - r^2 at midpoint between Eastern candidate (x+1, y) and South-Eastern candidate (x+1, y-1). Starting at (0, r) with p_0 = 1 - r, if p_k < 0, next point is (x+1, y) and p += 2x + 1; if p_k >= 0, next point is (x+1, y-1) and p += 2(x - y) + 1. Generates 8-way symmetry points across all 8 octants: (+-x, +-y) and (+-y, +-x).

5.4 Cohen-Sutherland Line Clipping Algorithm

Implemented in graphics.py:cohen_sutherland(). Encodes endpoint positions using a 4-bit region code [TOP, BOTTOM, RIGHT, LEFT]. If (c0 | c1) == 0, the segment is trivially accepted. If (c0 & c1) != 0, both points share an outside region and the segment is trivially rejected. Otherwise, it solves parametric edge intersections and recalculates outcodes until accepted or rejected.

5.5 Breadth-First Search (BFS) Grid Pathfinding

Implemented in levels.py:Level.find_bfs_path(). Uses a FIFO queue to explore unweighted grid corridor cells in order of increasing distance. Guarantees finding the minimal step path without traversing solid wall blocks.

CHAPTER 6: SYSTEM DESIGN AND ARCHITECTURE

6.1 Architecture & State Machine

The project employs a clean modular architecture separating low-level graphics (graphics.py, matrix.py), level carving and AI (levels.py), game state physics (game.py), and window management (main.py). The state machine manages transitions between title, gameplay, pause, level clear, victory, game over, and the standalone Computer Graphics Laboratory mode.

6.2 Game Loop & 60 FPS Synchronization

The game loop in main.py executes at 60 FPS using clock.tick(60). Each frame: (1) Input events are polled; (2) Physics, collisions, fog of war, and enemy AI update; (3) Scene geometry is clipped to the torch window and rendered; (4) pygame.display.flip() executes double buffering to present the completed frame smoothly.

CHAPTER 7: STEP-BY-STEP CODE EXPLANATION

7.1 Execution Flow Walkthrough

1. Startup: main.py initializes Pygame, creates an 800x600 window, and initializes Game(screen).
2. Menu Display: game.draw_title() renders stardust, crystal showcase, and dual action buttons.
3. Level Loading: game.new_game() instantiates Level(0), computing start/exit and entities.
4. Movement & Collision: game.update() integrates velocity; _collide() pushes player out of wall cells.
5. Abilities: Dash executes stepped bursts with afterimages; Shield absorbs hits completely without life loss.
6. Rendering: Renderer clips maze walls to the torch light and renders entities via affine transforms.

CHAPTER 8: GAMEPLAY FEATURES & ZERO-TIMER DESIGN

8.1 Action-Based Mechanics

In strict adherence to design requirements, EmberVault contains zero countdown timers or time limits. Directional Dash recharges every 35 paces walked through corridors. Energy Shield recharges upon collecting a Violet Aegis crystal or purging 2 wraiths. Cyan Surge crystals grant 5 Tri-Shot spread arrow volleys rather than a temporary timer window.

8.2 Caverns & Enemy Types

Features 5 calibrated caverns with distinct torch radii (155px down to 80px) and unique starting chambers. Enemies include Scout wraiths (patrolling corridors and chasing nearby players) and Stalker wraiths (actively hunting the player across the maze using BFS corridor pathfinding).

CHAPTER 9: USER INTERFACE AND CONTROLS

9.1 Interface Layout

The user interface features a high-contrast dark navy HUD showing cavern name, keys (0/1), score, shield status, dash gauge, heart indicators, and live Vault Radar minimap. Verified controls include WASD/Arrows for movement, SPACE/F for shooting, LSHIFT/X for dash, E for shield, G for Cartesian grid overlay, L for DDA/Bresenham toggle, C for Cohen-Sutherland/Liang-Barsky toggle, P for pause, and H for Field Guide.

CHAPTER 10: TESTING AND VERIFICATION

10.1 Verification Methodology & Results

Testing was conducted using an automated test harness (test_game.py) under a headless dummy video driver. All 16 unit and integration test cases passed with a 100% success rate, confirming mathematical correctness of Bresenham, DDA, Midpoint Circle, and Cohen-Sutherland clipping, as well as robust physics, ability recharges, and BFS AI navigation.

CHAPTER 11: RESULTS AND DISCUSSION

11.1 Observations & Discussion

The software successfully renders smooth 60 FPS graphics with rich visual aesthetics (radiant cyan torch glow, high-contrast walls, distinct pixel-art wraiths, and responsive particle feedback). Running low-level rasterization in pure Python allows complete mathematical transparency for educational viva defense.

CHAPTER 12: CONCLUSION AND FUTURE SCOPE

12.1 Conclusion & Academic Learnings

EmberVault demonstrates that engaging gameplay and rigorous computer graphics education can be combined without relying on commercial black-box engines. Future improvements include implementing hardware-accelerated shaders with PyOpenGL, adding dynamic 2D raycasted shadows, and expanding the maze into a 3D raycaster view.

CHAPTER 13: REFERENCES

13.1 Academic Citations

1. Hearn, D., Baker, M. P., & Carithers, W. (2014). Computer Graphics with OpenGL (4th ed.). Pearson Education.
2. Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F. (1995). Computer Graphics: Principles and Practice (2nd ed.). Addison-Wesley.
3. Bresenham, J. E. (1965). 'Algorithm for computer control of a digital plotter.' IBM Systems Journal, 4(1), 25-30.
4. Cohen, D., & Sutherland, I. E. (1968). 'Line clipping algorithms.' Newman and Sproull.
5. Liang, Y. D., & Barsky, B. A. (1984). 'A new concept and method for line clipping.' ACM TOG, 3(1), 1-22.
6. Pygame Community. (2024). Pygame Documentation (v2.6). https://www.pygame.org/docs/
7. Cormen, T. H., et al. (2009). Introduction to Algorithms (3rd ed.). MIT Press.